

6 Network Communities

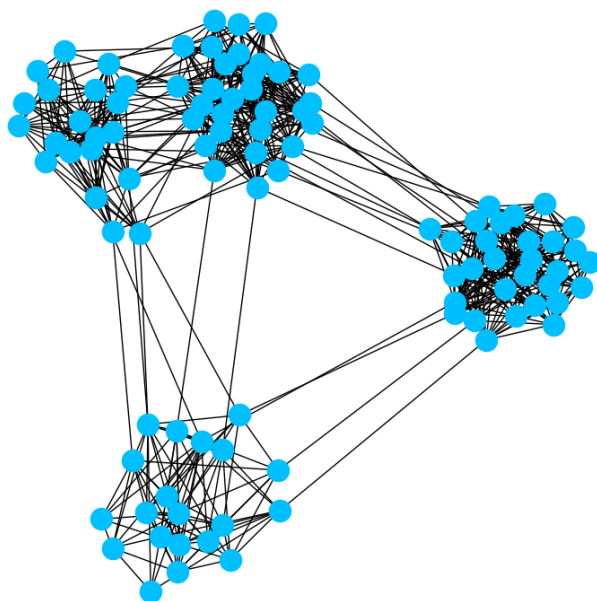


Figure 1: A modular network.

The force-directed layout of the network shown in Figure 1 suggests that this network consists of 4 strongly connected parts which are weakly connected to each other. Such a network is said to be **modular** and finding a node labelling that reflects the modular structure is called **community detection**. In this section we study **hard partitions**, where each node is a member of one and only one community. There are methods which attempt to assign soft partitions or overlapping communities which we will not discuss.

Reading

[1] Chapter 9, [2] Chapter 11.

6.1 Newman-Girvan Algorithm

A method which is somewhat similar to the KL algorithm is the **Girvan-Newman** (GN) algorithm. This relies on betweenness centrality which we have already discussed, but applied to edges. Define **edge betweenness**

$$b_e = \sum_{jk} \frac{n_{jk}(e)}{n_{jk}} \quad (1)$$

where n_{ij} is the number of shortest paths between i and j and $n_{ij}(e)$ is the number of such paths that use the edge e . The GN algorithm consists of the following steps

1. Calculate the edge betweenness of every edge.
2. Remove the edges with the highest betweenness.
3. Recalculate the betweenness.
4. Repeat step 2 and 3 until there are no more edges.

Note in step 3, not all the edges need to be updated, only those for which $n_{ij}(e)$ was non-zero. When enough edges are removed to disconnect part of the network then the GN algorithm outputs the nodes in each newly disconnected part.

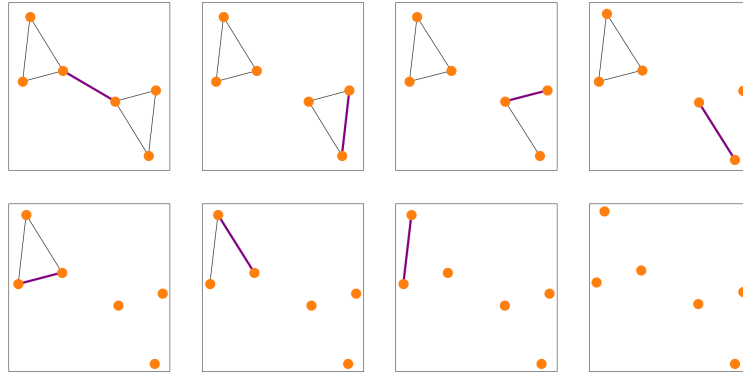


Figure 2: GN algorithm on a small network. Edge with max betweenness is highlighted in each panel.

This process is slow, but generally produces reasonable communities, Figure 2 shows the algorithm for a small network. The GN algorithm is an example of **hierarchical clustering** and the output is often expressed as a **dendrogram** which shows how the network breaks apart as the algorithm proceeds.

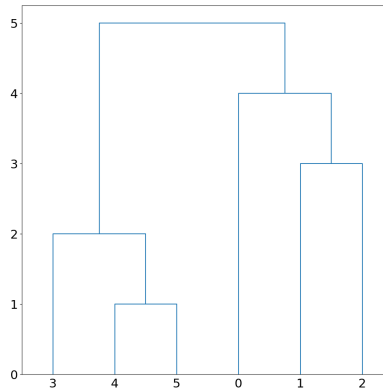


Figure 3: GN dendrogram for the GN process in Figure 1.

Figure 3 shows an example. To read this it is somewhat simpler to go from the bottom up, nodes 4 and 5 join at step 1 to make the community $\{4, 5\}$, then this community joins node 3 at the next step to make $\{3, 4, 5\}$. The same thing happens on the right to make $\{0, 1, 2\}$ and then they join to contain all the nodes in the network at the top.

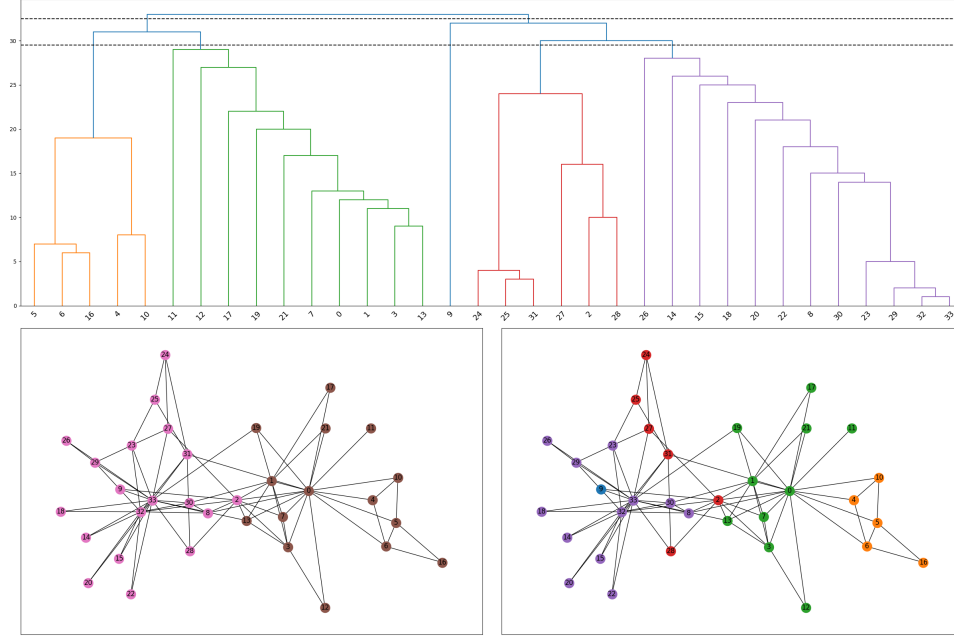


Figure 4: GN dendrogram and some example partitions. The top dashed line gives the partition on the left and the bottom gives the right hand one.

When the GN algorithm is applied to a more complex network we obtain various partitions, as in Figure 4 and we need some way to determine which one is the ‘best’. We will discuss one popular method in the next section.

6.2 Modularity

Modularity is a metric for measuring how well a given labelling does splitting the network into communities. The basic idea is that a network can often have strong degree correlations i.e. highly connected nodes are connected to each other, and we have to look for community structure *on top* of this. We therefore compare our actual network to one with the same degree sequence (i.e. the configuration model). We define the **modularity**

$$Q = \frac{1}{2L} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2L} \right) \delta(c(i), c(j)) \quad (2)$$

Here c is the **labelling function** which you can think of as colouring the nodes of the network.

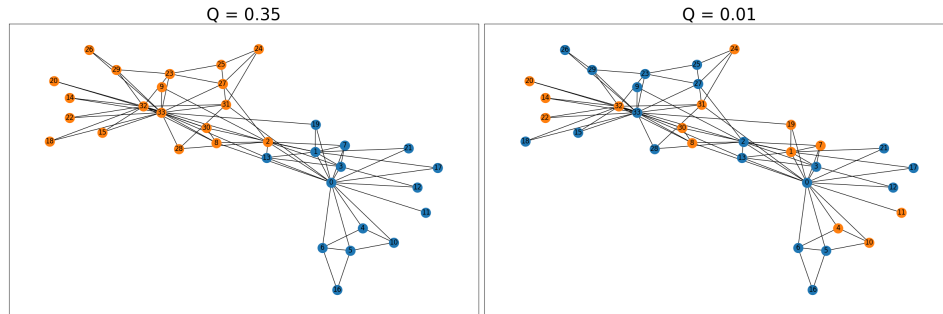


Figure 5: Modularity value of some example partitions.

Figure 5 shows that modularity is higher for intuitively good partitions, so if we can design an algorithm to maximise it then we will have a ‘good’ partition of the network. This optimisation means trying to pick the labelling function c which makes Q as large as possible on a given network. One simple way to do this for a partition into two parts is analogous to the KL algorithm - Compute the change in modularity from swapping every vertex into the other group and choose the one which most decreases or least increases modularity and follow the same steps as the KL algorithm until convergence. Basically we can swap minimising the cut with maximising the modularity. However as with the KL algorithm, this can be quite slow and we need something faster.

6.3 Louvain Algorithm

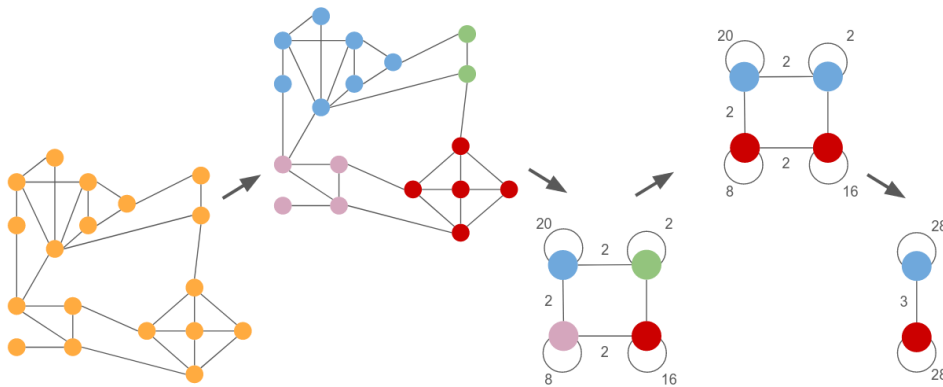


Figure 6: Louvain algorithm key steps.

A very popular algorithm for maximising modularity is known as the **Louvain** algorithm. The general idea is illustrated in Figure 6.

First, all the nodes are given a unique label i.e. they are in their own community. For every node we consider the change in modularity from changing the label of that node to the label of one of its neighbours. We choose the move that increases the modularity the most and, if none of the moves increases Q , then we leave the node in its original community. We do this for all nodes until the modularity stops increasing.

We then compute the **induced graph**

$$\tilde{A}_{ab} = \sum_{i \in a} \sum_{j \in b} A_{ij} \quad (3)$$

This is the network where all nodes with the same label are aggregated into one ‘super-node’ and the number of links between communities a and b is the sum of the links between nodes within those communities. Note the induced graph has self-loops and multi-edges (represented as edge weights) but this doesn’t cause any problem for the algorithm or for modularity. We then repeat this process of optimizing Q and aggregating until there are no more possible moves which increase modularity.

While this algorithm is fast there are numerous theoretical issues with it. You can use it, but make sure you know what you are doing!

6.4 Stochastic Block Model

Modularity has a number of known issues, some of which you will explore in the exercises. An alternative method is based on the **stochastic block model**(SBM). This is a very flexible model for generating networks with community structure (or any other structure you like). The basic idea is to find the parameters of the SBM which are most likely to have generated your data. These parameters include node labels, so in this way the network is ‘fit’ to the SBM.

The SBM generates networks with N nodes given a node labelling function c and a matrix ω of edge probabilities. For example, if we have a labelling function that gives the nodes of the network labels 0 or 1 and

$$\omega = \begin{pmatrix} p & q \\ q & r \end{pmatrix}$$

the SBM with matrix ω connects nodes with the 0 label together with probability p , joins nodes with the 1 label with probability r and joins 0 to

1 nodes with probability q . When ω is diagonal the model is called the **planted partition model**.

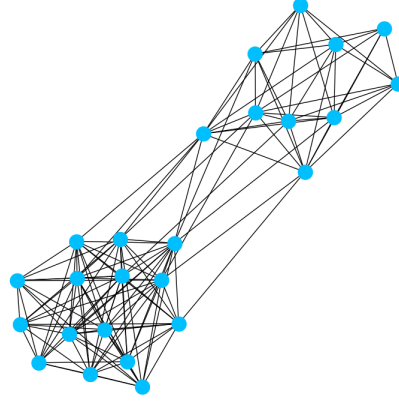


Figure 7: SBM network.

Figure 7 shows an SBM network with 15 nodes with label 0 and 10 with label 1 and

$$\omega = \begin{pmatrix} 0.8 & 0.1 \\ 0.1 & 0.8 \end{pmatrix}$$

This is reminiscent of an ER graph, but with community structure. Like the ER graph, it is not very realistic, in particular it doesn't have the right kind of degree distribution. For community detection the so-called **degree corrected Stochastic Block Model** (dc-SBM) is used instead.

In the standard SBM just described the expected value for an edge weight, A_{ij} , is

$$P(A_{ij}) = \frac{(\omega_{c(i)c(j)})^{A_{ij}}}{A_{ij}!} e^{-\omega_{c(i)c(j)}}$$

This is a Poisson distribution with mean $\omega_{c(i)c(j)}$. With some, not so simple, maths this can be used to write down the probability to generate the observed network, given the parameters c, ω . Community detection then consists of choosing c and ω which maximise the probability of observing the target network.

The degree corrected version changes the probability matrix ω_{ij} to $\theta_i \omega_{ij} \theta_j$ for some parameters θ . Again, some not so easy maths, shows that the maximum likelihood values of θ_i are just the degrees k_i . You can find all the

gory details in [3]¹. These methods are becoming popular, they are usually discussed as a ‘Bayesian’ approach to community detection, because they involve maximising a model likelihood, sometimes with a prior and are somewhat more robust than standard modularity maximisation, though often quite a bit slower on large networks.

6.5 Label Propagation

A different, and in many ways much simpler, approach to community detection is **label propagation**. The basic idea is to give all the nodes a unique label and try to update every node to reach consensus with its neighbours i.e. every node should have the same label as most of its neighbours.

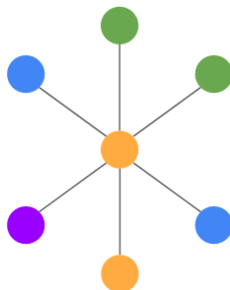


Figure 8: Star graph labelled with different colours.

There will be sometimes be ties, for example the central node in Figure 8 has 2 green neighbours and 2 blue ones. Usually these are broken by randomly picking one of the majority labels. There are two main ways to implement label propagation. First is the **synchronous** algorithm. Let c_t be the labelling function at step t then

1. At step zero give each node a unique label $c_0(i) = i$.
2. At step t update the labels of each node with the most common label of their neighbours using c_{t-1} , breaking ties at random.
3. Perform step 2 as many times as necessary until the labelling function stops changing.

¹ <https://arxiv.org/pdf/1008.3926>

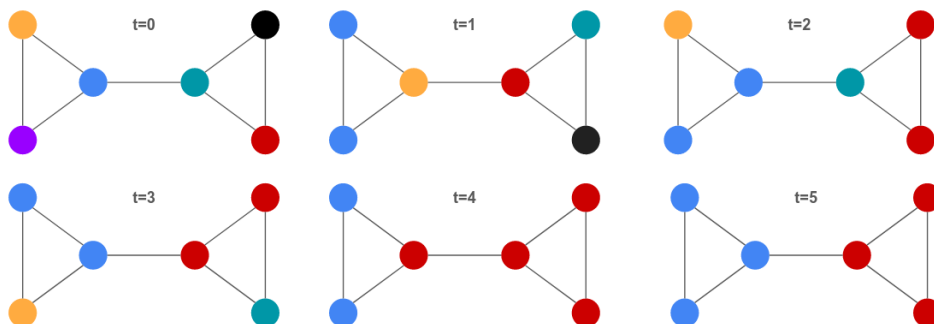


Figure 9: Label propagation.

Figure 9 shows one possible unfolding of synchronous label propagation on a small graph. The final configuration is a fixed point, all nodes have the same label as the majority of their neighbours. This algorithm works well in some cases and is also extremely parallelisable, since once we have c_{t-1} we can update every node at the same time. However the fact the above process finished in only six steps was quite lucky, this algorithm is quite prone to cyclically oscillating between labels. An alternative algorithm, less prone to oscillation is **asynchronous** label propagation.

1. At step zero give each node a unique label $c(i) = i$.
2. At step t generate a random permutation of the vertices V_p
3. **for** i **in** V_p :
Set $c(i)$ to be the majority label of the neighbours of i , breaking ties at random.
4. Perform steps 2 and 3 as many times as necessary until the labelling function stops changing.

Rather than update all the nodes at once, this updates them one at a time in some random order. This version is less prone to oscillate but cannot be parallelised as easily.

6.6 Walktrap

A community detection algorithm that combines many of the features we've discussed - hierarchical clustering, modularity, propagation as well as being a nice transition into the next topic about spreading on networks is the **walktrap algorithm**. The basic idea is to imagine walking around the

nodes of the network, following links at random. Such a walk will spend a long time ‘trapped’ in densely connected parts of the network.

Let P_{ij} be the probability of going from i to j via a random walk of length 1. We have

$$P_{ij} = \frac{A_{ij}}{k_i} \quad (4)$$

i.e. 0 chance if they’re not connected, 1 in k_i chance if they are. It is also useful to write this as a matrix equation

$$P = D^{-1}A$$

Where D is the matrix with the degrees on the diagonal we saw when we introduced the graph Laplacian. By repeatedly multiplying P we get P_{ij}^t , the probability to go from i to j in t steps. We make the following observations

- If nodes i and j are in the same community then P_{ij}^t will be high
- A walker has a higher probability to go to higher degree nodes
- Two nodes in the same community should ‘see’ the rest of the network similarly i.e. $P_{ik}^t \simeq P_{jk}^t$

For a community C (a set of nodes with the same label) define

$$P_{Cj}^t = \frac{1}{|C|} \sum_{i \in C} P_{ij}^t \quad (5)$$

as the probability to go from some node in C to node j in time t . We then define the distance between communities C and D as

$$r_{CD} = \sqrt{\sum_j \frac{(P_{Cj}^t - P_{Dj}^t)^2}{k_j}} \quad (6)$$

This gives a distance between communities. The Walktrap algorithm tries to minimise this distance. The steps are

1. Start every node in its own community.
2. Merge communities to minimise

$$\sigma = \frac{1}{N} \sum_C \sum_{i \in C} r_{iC}^2$$

which is the sum of squared distances between all nodes and their communities.

The key thing about this is that there is a fast way to update σ and the distances r_{Cj} after every merge. Like the GN algorithm it produces many partitions, and we can choose the best one by some criteria like maximum modularity. Another thing which helps the speed of this algorithm is that t , the number of steps taken by the random walker, cannot be too high, otherwise the algorithm doesn't work because the high degree nodes suck up all the random walkers. Values of $2 \leq t \leq 5$ are suggested by [4].

References

- [1] Márton Pósfai and Albert-László Barabási. *Network science*. Cambridge University Press, 2016.
- [2] Mark Newman. *Networks*. Oxford university press, 2018.
- [3] Brian Karrer and Mark EJ Newman. Stochastic blockmodels and community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 83(1):016107, 2011.
- [4] Pascal Pons and Matthieu Latapy. Computing communities in large networks using random walks. In *Computer and Information Sciences-ISCIS 2005: 20th International Symposium, Istanbul, Turkey, October 26-28, 2005. Proceedings 20*, pages 284–293. Springer, 2005.